

RED TEAM LEARNING PATH

MODULE 13

API SECURITY

REST/JSON attack surface, authorization, business logic, inventory & GraphQL



Isolated local API training architecture

Authorized-lab only

Semua request aktif dalam module ini hanya untuk localhost/container milik sendiri atau deliberately vulnerable training platform seperti PortSwigger Web Security Academy. Menemukan endpoint API publik tidak berarti endpoint tersebut boleh diuji.

Field	Value
Primary OS	Omarchy / Arch Linux
Prerequisite	Modules 00-12
Core tools	Burp Suite, curl, jq, Docker, Python
Local target	127.0.0.1:8130
Outcome	Mampu menilai API secara sistematis dari inventory sampai finding + remediation

1. Learning Objectives

- [] Membedakan **authentication**, **object-level**, **property-level**, dan **function-level authorization** sebagai empat pertanyaan yang berbeda.
- [] Membangun endpoint inventory dari traffic observed + dokumentasi OpenAPI, lalu menandai gap di antara keduanya.
- [] Menguji BOLA/IDOR tanpa mengganggu UUID atau ID yang "susah ditebak" sebagai kontrol akses.
- [] Menguji broken object property-level authorization (excessive data exposure + mass assignment).
- [] Menguji broken function-level authorization pada operasi administratif.
- [] Menilai resource consumption & rate limit secara low-volume, tanpa stress-test.
- [] Memahami sensitive business flows sebagai kelas risiko tersendiri (API6), bukan sekadar bug teknis.
- [] Memahami API versioning, deprecated/shadow endpoint, dan kenapa inventory adalah kontrol keamanan.
- [] Menilai JWT/API token secara kontekstual — decode bukan verify, format bukan trust.
- [] Mengenal metodologi dasar GraphQL testing (schema, resolver authorization, introspection).
- [] Menulis finding note yang reproducible: request, response delta, root cause, impact, remediation.

2. Mental Model: Empat Pertanyaan yang Sering Tercampur

API security gagal paling sering bukan karena bug kriptografi, tapi karena empat pertanyaan berikut dicampur jadi satu ("token valid = boleh semua"):

Lapisan	Pertanyaan	Contoh kegagalan
Authentication	Siapa caller-nya?	Token lemah/dipalsukan, login bypass
Object-level authorization	Boleh menyentuh object <i>ini</i> ?	BOLA/IDOR — endpoint benar, object salah
Property-level authorization	Boleh baca/tulis field <i>ini</i> pada object itu?	Excessive data exposure, mass assignment (role ikut ter-update)
Function-level authorization	Boleh memanggil operasi <i>ini</i> ?	User biasa memanggil endpoint admin karena authorization server-side tidak dicek

Untuk setiap endpoint yang lo temukan, selalu tanyakan kelima hal ini sebelum menyimpulkan apa pun:

- Siapa caller-nya (identity apa yang dipakai)?
- Object apa yang disentuh?
- Property mana yang boleh dibaca/diubah?
- Function/operation apa yang dipanggil?
- Resource apa yang dikonsumsi, dan business invariant apa yang harus tetap benar?

Think like a tester.

Jangan mulai dari "endpoint ini vulnerable apa enggak". Mulai dari "siapa yang authorized melakukan apa terhadap apa", lalu buktikan server benar-benar menegakkannya.

3. OWASP API Security Top 10 (2023)

Risk	Inti masalah	Dibuktikan di lab lewat
API1 — BOLA	Caller boleh endpoint-nya, tapi tidak boleh object yang dipilihnya	GET /api/v1/orders/{id} — tidak ada owner check
API2 — Broken Authentication	Identity/token handling gagal sehingga caller bisa mengambil identitas lain	Login flow & token mapping di lab (observasi konsep, bukan crack)
API3 — BOPLA (Broken Object	Property sensitif terbaca atau bisa	GET /api/v1/me (exposure

Property-Level Authorization)	ditulis tanpa authorization yang benar	internal_note), PATCH /api/v1/me (mass assignment role)
API4 — Unrestricted Resource Consumption	Request bisa menghabiskan CPU/memori/bandwidth/biaya tanpa guardrail	GET /api/v1/report?limit= — parameter mengontrol ukuran response
API5 — BFLA (Broken Function-Level Authorization)	User bisa memanggil function yang seharusnya untuk role lain	GET /api/v1/admin/stats dengan token non-admin
API6 — Unrestricted Access to Sensitive Business Flows	Flow valid disalahgunakan secara otomatis/berulang hingga merugikan bisnis	POST /api/v1/coupons/redeem dipanggil berulang tanpa batas per-user
API7 — Server-Side Request Forgery	API mengambil URI dari user tanpa validasi destination	Dipraktikkan lebih dalam di Module 12
API8 — Security Misconfiguration	Debug mode, CORS longgar, verbose error, default credential, dll.	Observasi header & error response lab
API9 — Improper Inventory Management	Versi lama/shadow endpoint tidak terinventarisasi	GET /api/v0/debug/users — ada di server, tidak ada di openapi.json
API10 — Unsafe Consumption of APIs	Response third-party API dipercaya terlalu tinggi	Dibahas konseptual (§12) — lab ini tidak memanggil API eksternal
OWASP 2023 menggabungkan <i>Excessive Data Exposure</i> lama dan <i>Mass Assignment</i> lama menjadi API3, karena root cause-nya sama: authorization pada property object tidak ditegakkan dengan benar — terlepas dari apakah arah datanya keluar (exposure) atau masuk (assignment).		

4. Lab Architecture & Containment

Lab menggunakan dummy in-memory data (alice, bob, admin) dan deliberately-broken authorization. Tidak ada external callback, credential nyata, atau persistent database — restart container berarti state reset total.

```
Container: module13-api-lab (Python http.server, single file app.py)
Bind:      127.0.0.1:8130 (tidak dipublish ke LAN/internet)
Hardening: --cap-drop ALL --security-opt no-new-privileges:true
           --read-only --tmpfs /tmp:rw,noexec,nosuid,size=16m
           tidak ada host volume mount
```

Vulnerability-nya sengaja berada di **application authorization / business logic**, bukan container/host escape — konsisten dengan tujuan module: lo belajar menilai *access control*, bukan mencari container breakout.

Setup

```
cd modules/13-api-security/labs
./workspace-init.sh          # buat struktur evidence: 00-scope .. 06-summary
./api-lab.sh build
./api-lab.sh create
curl -s http://127.0.0.1:8130/ | jq .
```

Perintah lab yang tersedia: build, create/start, status, logs, cleanup/stop.

```
./api-lab.sh cleanup        # wajib dijalankan setelah selesai
```

5. Hands-on 5.1 — Establish Two Identities

Authorization testing butuh **minimal dua identity** — kalau cuma satu, lo tidak bisa membedakan "endpoint memang kosong" dari "server tidak mengecek siapa yang bertanya".

```
curl -s -X POST http://127.0.0.1:8130/api/v1/login \
-H 'Content-Type: application/json' \
-d '{"username":"alice","password":"training123"}' | jq .

curl -s -X POST http://127.0.0.1:8130/api/v1/login \
-H 'Content-Type: application/json' \
-d '{"username":"bob","password":"training123"}' | jq .
```

Lab training token sudah dalam bentuk tetap (token-alice, token-bob, token-admin) supaya lo bisa fokus ke authorization logic, bukan menebak token. Simpan token ini hanya di catatan lokal/workspace — jangan pernah perlakukan pola ini sebagai hal yang boleh dibawa ke sistem nyata.

Alternatif pakai helper client bawaan module (otomatis menolak target non-loopback):

```
python labs/api-client.py POST http://127.0.0.1:8130/api/v1/login \
--json '{"username":"alice","password":"training123"}'
```

6. API Recon & Documentation Discovery

Recon API dimulai dari traffic yang benar-benar dipakai aplikasi (method, path pattern, content-type, error message, referensi di JS/mobile client) **digabung** dengan dokumentasi machine-readable seperti OpenAPI. Dokumentasi adalah peta, bukan izin — dan peta itu bisa salah atau tidak lengkap.

```
mkdir -p module13-api-security/01-recon

curl -s http://127.0.0.1:8130/openapi.json \
-o module13-api-security/01-recon/openapi.json

python labs/openapi-inventory.py \
module13-api-security/01-recon/openapi.json \
--output module13-api-security/01-recon/endpoints.csv

column -s, -t module13-api-security/01-recon/endpoints.csv
```

Recon gap by design.

Lab ini sengaja punya endpoint deprecated (/api/v0/debug/users) yang **tidak tercantum** di openapi.json. Kalau inventory lo cuma mengandalkan dokumentasi, endpoint itu akan luput — lihat \$10 Inventory Management untuk cara menemukannya dari observed traffic.

7. API1:2023 — BOLA / IDOR (Object-Level Authorization)

BOLA terjadi ketika endpoint menerima object identifier dan server **tidak memverifikasi** caller memang berhak atas object tersebut. Baik ID sequential (101, 102) maupun UUID sama-sama bukan kontrol akses — UUID cuma membuat identifier lebih susah ditebak, bukan membuatnya "authorized by design".

```
# Alice buka order miliknya sendiri
curl -s http://127.0.0.1:8130/api/v1/orders/101 \
-H 'Authorization: Bearer token-alice' | jq .

# Token Alice yang sama, tapi order milik Bob
curl -s http://127.0.0.1:8130/api/v1/orders/102 \
-H 'Authorization: Bearer token-alice' | jq .
```

Kalau kedua request berhasil mengembalikan data, **jangan** tulis root cause sebagai "ID mudah ditebak". Root cause sebenarnya: *object-level authorization tidak divalidasi setelah object ditemukan* — server menjawab "object 102 ada" tanpa pernah bertanya "apakah alice boleh melihat object 102 ini".

Remediation: load object dulu, baru authorize berdasarkan kombinasi (authenticated principal, requested action, object policy). Untuk model bisnis yang lebih kompleks (shared resource, organisasi multi-tenant, delegated access), perbandingan sederhana `user_id == owner_id` sering tidak cukup — authorization harus mengikuti policy yang sama dengan yang dipakai UI.

8. API3:2023 — BOPLA / Mass Assignment (Property-Level Authorization)

API3 mencakup **dua arah** masalah pada property level:

Arah	Nama lama	Contoh
------	-----------	--------

Keluar (response)	Excessive Data Exposure	Response membawa field yang client tidak butuh, mis. <code>internal_note</code>
Masuk (request)	Mass Assignment	Request mengizinkan client menulis field yang seharusnya server-controlled, mis. <code>role</code>

8.1 — Excessive data exposure

```
curl -s http://127.0.0.1:8130/api/v1/me \
-H 'Authorization: Bearer token-alice' | jq .
```

Catat field apa saja yang muncul di response. Kalau ada field seperti `internal_note` yang tidak pernah ditampilkan UI dan tidak dibutuhkan client, itu sudah merupakan finding — data minimization adalah bagian dari *API design*, bukan sekadar kerapihan kosmetik.

8.2 — Mass assignment ke field privileged

```
curl -s -X PATCH http://127.0.0.1:8130/api/v1/me \
-H 'Authorization: Bearer token-alice' \
-H 'Content-Type: application/json' \
-d '{"role":"admin"}' | jq .
```

Lalu bandingkan state sebelum/sesudah dengan `json-diff.py` — berguna supaya finding lo punya bukti delta yang eksplisit, bukan cuma klaim:

```
curl -s http://127.0.0.1:8130/api/v1/me -H 'Authorization: Bearer token-alice' \
> module13-api-security/03-responses/me-before.json

curl -s -X PATCH http://127.0.0.1:8130/api/v1/me \
-H 'Authorization: Bearer token-alice' -H 'Content-Type: application/json' \
-d '{"role":"admin"}' > /dev/null

curl -s http://127.0.0.1:8130/api/v1/me -H 'Authorization: Bearer token-alice' \
> module13-api-security/03-responses/me-after.json

python labs/json-diff.py \
module13-api-security/03-responses/me-before.json \
module13-api-security/03-responses/me-after.json
```

Output `json-diff.py` menunjukkan key mana yang changed, sehingga di finding note lo bisa menulis *"field `role` berubah dari `user` menjadi `admin` akibat request PATCH yang sama"* — bukti yang jauh lebih kuat daripada screenshot tunggal.

Root cause.

Generic object-binding/patch logic menerima property `role` langsung dari client. **Fix:** definisikan *explicit writable-field allow-list* per operation (bukan blacklist), dan terapkan authorization per-property — bukan hanya per-endpoint.

9. API5:2023 — BFLA (Function-Level Authorization)

BFLA terjadi ketika caller yang **terautentikasi dengan sah** bisa memanggil operasi administratif/sensitif karena server tidak mengecek *function-level* authorization — biasanya karena tim developer mengandalkan UI yang menyembunyikan tombol admin, bukan enforcement di server.

```
curl -s http://127.0.0.1:8130/api/v1/admin/stats \
-H 'Authorization: Bearer token-bob' | jq .
```

Kalau request ini berhasil dengan token Bob (bukan admin), **jangan laporkan sebagai authentication failure** — token Bob valid, identitasnya benar. Yang gagal adalah authorization terhadap fungsi administratif itu sendiri.

Remediation: enforce *deny-by-default* authorization di server untuk setiap function/route berdasarkan role/permission yang tervalidasi dari sesi, bukan dari asumsi "user biasa tidak tahu endpoint-nya".

10. API4:2023 — Resource Consumption & Rate Limits

API4 bukan cuma soal *requests-per-second*. Parameter seperti pagination size, export size, GraphQL query depth, ukuran file upload, pemanggilan third-party API yang mahal, atau OTP/SMS/email juga bisa menghabiskan CPU, memori, bandwidth, atau **biaya** — tanpa pernah menyentuh rate limiter klasik sama sekali.

```
curl -s 'http://127.0.0.1:8130/api/v1/report?limit=5' \
-H 'Authorization: Bearer token-alice' | jq '{requested,returned}'

curl -s 'http://127.0.0.1:8130/api/v1/report?limit=1500' \
-H 'Authorization: Bearer token-alice' | jq '{requested,returned}'
```

Jangan stress-test.

Cukup observasi behavior dengan request kecil. Lab punya safety cap internal di 2000 baris (lihat header X-Training-Safety-Cap). Pada assessment production, volume/rate testing **harus** eksplisit tertulis di Rules of Engagement karena berisiko DoS atau membengkakkan biaya cloud klien.

Observasi throttling signal dengan low-volume request (dibatasi helper ke maksimum 12 request):

```
python labs/rate-check.py \
'http://127.0.0.1:8130/api/v1/report?limit=1' \
--token token-alice \
--count 8
```

Perhatikan status, retry_after, dan latency (ms) tiap request. Delapan request **bukan bukti** API tahan/tidak tahan DoS — ini cuma exercise melihat apakah ada signal basic throttling (Retry-After, status 429, latency naik) sama sekali.

11. API6:2023 — Sensitive Business Flows

API6 membahas functionality yang **secara teknis valid** tapi berbahaya kalau dieksekusi otomatis/berulang-ulang: redeem coupon, reservasi inventory, posting, signup, referral bonus, checkout, pembelian tiket, dan sejenisnya.

```
for i in 1 2 3; do
  curl -s -X POST http://127.0.0.1:8130/api/v1/coupons/redeem \
  -H 'Authorization: Bearer token-bob' \
  -H 'Content-Type: application/json' \
  -d '{"code":"WELCOME"}' | jq '{redeemed,count_for_user}'
done
```

Pertanyaan bisnis yang harus lo jawab di finding: apakah coupon ini seharusnya *sekali per user*, *sekali per akun/device*, atau memang *reusable by design*? Finding security yang baik butuh **expected business invariant** yang eksplisit — bukan sekadar observasi "endpoint bisa dipanggil tiga kali berturut-turut", karena itu bisa saja memang perilaku yang diinginkan.

12. API9:2023 — Inventory & Version Management

API9 muncul ketika organisasi kehilangan visibility atas versi, host, environment, dokumentasi, dan endpoint deprecated/shadow. API versi lama sering tertinggal dengan authorization atau logging yang lebih lemah dibanding versi aktif — tapi tetap *reachable*.

```
curl -i -s http://127.0.0.1:8130/api/v0/debug/users \
-H 'Authorization: Bearer token-alice' | sed -n '1,25p'
```

Endpoint v0 ini **tidak** tercantum di openapi.json (lihat §6) — inventory yang benar harus mencakup *route yang ter-deploy*, bukan hanya yang ter-dokumentasi. Perhatikan juga header Deprecation: true sebagai sinyal eksplisit bahwa endpoint ini seharusnya sudah tidak dipakai, tapi tetap merespons.

13. JWT & API Tokens in Context

JWT cuma **satu format** untuk membawa claims. API juga bisa memakai opaque token (seperti di lab ini), API key, mTLS, session cookie, atau signed request — format token **tidak menentukan** model authorization-nya.

JWT refresher.

Payload JWT bisa dibaca siapa saja yang memegang tokennya (base64, bukan enkripsi). Trust baru ada setelah *signature/MAC* divalidasi **dan** claim (iss, aud, exp, nbf) diperiksa sesuai RFC 7519. **Decode ≠ verify**.

Lab ini sengaja memakai opaque training token (token-alice, bukan JWT) justru untuk menegaskan satu hal: BOLA, BOPLA, dan BFLA bisa terjadi **meskipun authentication token itu sendiri sama sekali tidak rusak**. Masalahnya selalu ada satu lapis setelah "token ini valid" — yaitu "apa yang boleh dilakukan token ini".

14. GraphQL API Testing (Pengantar)

GraphQL tetap API — tapi attack surface-nya berpusat pada **schema**, query/mutation, **resolver authorization**, introspection, aliases/batching, dan query cost. Schema visibility (introspection berhasil) **bukan** authorization control.

- Buka topik *GraphQL API vulnerabilities* di PortSwigger Web Security Academy.

- Mulai dari lab tentang menemukan endpoint GraphQL / introspection.
- Petakan query, variables, operationName, object types, dan mutation yang tersedia.
- Uji authorization pada level object/field sesuai objective lab — bukan sekadar "introspection aktif = vulnerable".
- Kalau lab memakai aliases/batching, catat bagaimana satu HTTP request bisa memicu banyak eksekusi resolver sekaligus.
- Tulis finding dengan root cause di **resolver/policy**, bukan "GraphQL enabled".

Guardrail.

Jangan menjalankan deep/recursive query atau high-volume batching ke API produksi tanpa izin eksplisit — query-complexity testing pada dasarnya adalah availability test.

15. API10:2023 — Unsafe Consumption of Third-Party APIs (Konsep)

API10 menyoroti trust berlebihan terhadap API pihak ketiga yang dikonsumsi aplikasi. Response third-party tetap **untrusted input** — bisa malformed, malicious, stale, overprivileged, atau berasal dari provider/dependency yang sudah ter-compromise.

Pertanyaan	Yang harus diverifikasi
Transport	HTTPS/TLS verification aktif, tidak ada insecure fallback
Authentication	Scope credential minimal, rotasi rutin, token tidak bocor ke log
Response schema	Type/size/schema divalidasi sebelum dipakai aplikasi
Redirects/URL	Destination divalidasi — hindari blind redirect / SSRF chain
Timeout/retry	Timeout wajar, retry budget terbatas, ada circuit breaker
Authorization	Data dari third-party tidak boleh melewati access-policy lokal begitu saja

16. Black-Box Practical Challenge

Jangan buka `labs/api-lab/app.py` dulu. Kerjakan murni lewat Burp/curl dan dokumentasi yang memang diekspos lab.

- [] Bangun endpoint inventory dan tandai mana yang *documented* vs *discovered-only*.
- [] Pakai identity Alice & Bob untuk menemukan minimal satu object-level authorization flaw.
- [] Temukan property yang seharusnya tidak bisa dibaca atau ditulis caller.
- [] Temukan function administratif dengan authorization lemah.
- [] Analisis satu resource/business-flow weakness **tanpa** high-volume testing.
- [] Temukan deprecated/shadow API version.
- [] Tulis minimal 4 finding lengkap: baseline, modified request, response delta, root cause, impact, remediation.

```
python labs/finding-note.py \
  module13-api-security/04-findings/bola-order.md \
  --title "Broken object-level authorization on orders" \
  --category "API1:2023 BOLA" \
  --endpoint "GET /api/v1/orders/{id}" \
  --evidence "Alice token can read Bob order 102." \
  --impact "A low-privilege user can read another user's order." \
  --remediation "Authorize requested order against caller server-side."
```

Setelah selesai:

```
./labs/api-lab.sh cleanup
```

17. Evidence & Reporting Discipline

Elemen evidence	Minimum yang dicatat
Identity	Role/user yang dipakai; redact token asli kalau report keluar workspace
Baseline	Request + response yang diharapkan (authorized)
Mutation	Satu ID/property/function/parameter yang diubah — single-variable change
Delta	Perbedaan status/body/side-effect dibanding baseline
Root cause	Object/property/function/business-rule check mana yang hilang
Impact	Efek data/aksi/bisnis nyata, sesuai scope — jangan dilebih-lebihkan
Remediation	Kontrol server-side yang konkret, sebut lokasinya (endpoint/handler)

Jangan paste secret asli.

Authorization header, API key, refresh token, signed URL, session cookie, dan webhook secret harus diperlakukan sebagai secrets. Repository publik hanya boleh memuat dummy/local token.

18. Knowledge Check

- Apa beda authentication dengan object-level authorization?
- Kenapa UUID tidak dengan sendirinya memperbaiki BOLA?
- Apa dua arah utama BOPLA, dan kenapa keduanya digabung di API3:2023?
- Kenapa "tombol admin disembunyikan di UI" bukan function-level authorization?
- Apa beda rate limit dengan resource limit?
- Kenapa API6 (sensitive business flow) membutuhkan konteks bisnis, bukan cuma observasi teknis?
- Apa risiko nyata dari API version lama yang tidak terinventarisasi?
- Kenapa dokumentasi OpenAPI bukan bukti lengkap seluruh endpoint yang ter-deploy?
- Kenapa men-decode JWT tidak sama dengan memverifikasinya?
- Apa yang wajib divalidasi saat mengonsumsi response dari third-party API?
- Kenapa GraphQL introspection yang aktif tidak otomatis berarti critical finding?
- Mengapa authorization testing butuh minimal dua identity berbeda?
- Kapan high-volume testing pada API harus dihentikan dan dieskalasi ke engagement lead?
- Apa evidence minimum yang cukup untuk sebuah BOLA finding?
- Seperti apa remediation mass assignment yang benar-benar production-ready (bukan sekadar blacklist field)?

19. Cheat Sheet

```
# Login & dapatkan token training
curl -s -X POST http://127.0.0.1:8130/api/v1/login \
  -H 'Content-Type: application/json' -d '{"username":"alice","password":"training123"}' | jq .

# Authenticated GET
curl -s http://127.0.0.1:8130/api/v1/me -H 'Authorization: Bearer token-alice' | jq .

# BOLA differential test
curl -s http://127.0.0.1:8130/api/v1/orders/101 -H 'Authorization: Bearer token-alice' | jq .
curl -s http://127.0.0.1:8130/api/v1/orders/102 -H 'Authorization: Bearer token-alice' | jq .

# Mass assignment probe
curl -s -X PATCH http://127.0.0.1:8130/api/v1/me \
  -H 'Authorization: Bearer token-alice' -H 'Content-Type: application/json' -d '{"role":"admin"}' | jq .

# BFLA probe
curl -s http://127.0.0.1:8130/api/v1/admin/stats -H 'Authorization: Bearer token-bob' | jq .

# Shadow endpoint
curl -i -s http://127.0.0.1:8130/api/v0/debug/users -H 'Authorization: Bearer token-alice' | sed -n '1,20p'

# Cleanup
./labs/api-lab.sh cleanup
```

20. Exit Criteria

- [] Bisa membangun endpoint inventory dari OpenAPI + observed traffic, dan menandai gap di antara keduanya.
- [] Bisa membedakan BOLA, BOPLA, BFLA, dan authentication flaw tanpa tertukar.
- [] Bisa menjalankan two-identity differential authorization test dan mendokumentasikan delta-nya.
- [] Bisa menguji writable/readable property secara minimal dan terkontrol.
- [] Bisa mengobservasi resource/rate behavior tanpa melakukan stress test.
- [] Bisa menjelaskan sensitive-business-flow vulnerability lengkap dengan expected invariant-nya.
- [] Bisa menemukan dan menilai deprecated/shadow API version.
- [] Bisa menjelaskan kenapa JWT adalah format token, bukan authorization control.
- [] Bisa mengikuti satu lab GraphQL Academy dan memetakan schema/resolver authorization.
- [] Bisa menulis finding API yang reproducible, dengan evidence yang cukup dan tanpa membocorkan secret.

21. Deliverables

- docx/Red-Team-Module-13-API-Security-0marchy.docx — versi editable
- pdf/Red-Team-Module-13-API-Security-0marchy.pdf — versi final untuk dibaca/distribusikan
- labs/ — local loopback-only API lab + helper utilities:
- workspace-init.sh — scaffold workspace evidence (00-scope ... 06-summary)
- api-lab.sh — build/create/status/logs/cleanup container lab
- api-client.py — HTTP client minimal yang menolak target non-loopback
- openapi-inventory.py — konversi openapi.json → CSV endpoint inventory
- rate-check.py — low-volume probe untuk melihat signal throttling
- json-diff.py — bandingkan dua snapshot JSON response (before/after)
- finding-note.py — generate finding note markdown terstruktur

22. References

- OWASP API Security Project — <https://owasp.org/projects/api-security-project>
- OWASP API Security Top 10 (2023) — <https://api-security.owasp.org/editions/2023/en/0x11-t10/>
- OWASP API1:2023 Broken Object Level Authorization — <https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/>

- OWASP API3:2023 Broken Object Property Level Authorization — <https://api-security.owasp.org/editions/2023/en/0xa3-broken-object-property-level-authorization/>
- OWASP API4:2023 Unrestricted Resource Consumption — <https://api-security.owasp.org/editions/2023/en/0xa4-unrestricted-resource-consumption/>
- OWASP API10:2023 Unsafe Consumption of APIs — <https://api-security.owasp.org/editions/2023/en/0xaa-unsafe-consumption-of-apis/>
- PortSwigger Web Security Academy — API Testing — <https://portswigger.net/web-security/learning-paths/api-testing>
- PortSwigger Web Security Academy — GraphQL — <https://portswigger.net/web-security/graphql>
- RFC 7519 — JSON Web Token — <https://www.rfc-editor.org/rfc/rfc7519.html>

Next module:

14-exploitation-fundamentals — exploit research, CVE/PoC triage, safe public-exploit review, konsep shell/payload, dasar Metasploit, file transfer, evidence, dan target training yang memang sengaja vulnerable.